Step 3 — Add the Resource and the Prompt — Solution

> Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 3: the code here is tiny; the point is the architectural decision. A plain listing of notes *could* be built as a third Tool, and a summary prompt *could* just be a string you type by hand — but MCP gives you two other primitives built exactly for this: a read-only listing with nothing to act on belongs as a **Resource**, and a reusable, parameterized template belongs as a **Prompt**, not hardcoded inside one app. Getting that split right is the actual skill this step teaches. **If not:** cramming everything into Tools is the real mistake real MCP servers make — a client that only wants to *read* your notes shouldn't have to go through a Tool call, with its own approval model, just to do it.

After this step the project runs: `add_note` and `search_notes` are unchanged, and the server also exposes `notes://all` (Resource) and `summarize_notes` (Prompt). Files not shown here don't change.

Solution — a read-only Resource and a pure-template Prompt

Changed from Step 2: `db.py` gains `list_all_notes`; `server.py` gains the Resource and the Prompt. The two Tools don't change at all — they stay Tools, on purpose.

Story — `db.py` → `list_all_notes`: this is a plain `SELECT` of every row, ordered oldest first, that the Resource then formats for display. **If not:** the Resource function would build its own query inline, duplicating logic that belongs in `db.py`.

```
# src/db.py (new function — add alongside Step 2's functions)
def list_all_notes(path):
    conn = sqlite3.connect(path)
    rows = conn.execute(
        "SELECT id, text, created_at FROM notes ORDER BY id"
    ).fetchall()
    conn.close()
    return rows
```

Story — `server.py` → the Resource and the Prompt: `@mcp.resource("notes://all")` takes no arguments and has no side effects — it only ever hands back data to read, with an explicit `"No notes yet."` string when the store is empty rather than nothing at all. `@mcp.prompt()` returns the

text of a prompt template, filled in with the two date placeholders — the server never calls a model or reads the database itself here; it simply points the caller at the `notes://all` Resource for the actual content. A Resource's and a Prompt's docstrings matter to a client's UI exactly the way a Tool's does to the model. **If not:** a Prompt that quietly re-reads the database itself is the same mistake as a Tool with a vague description — it works today and confuses everyone the moment the server grows more complex.

```
# server.py (new pieces — add below the two Tools, from Step 2)
from src.db import list_all_notes     # new in Step 3

# new in Step 3: a read-only Resource — no arguments, no side effect
@mcp.resource("notes://all")
def all_notes():
    """A read-only listing of every note currently stored, oldest first."""
    rows = list_all_notes(DB_PATH)
    if not rows:
        return "No notes yet."
    lines = []
    for row in rows:
        lines.append(f"[{row[0]}] {row[1]} ({row[2]})")
    return "\n".join(lines)

# new in Step 3: a Prompt — a reusable template, not a summary
@mcp.prompt()
def summarize_notes(start_date, end_date):
    """A reusable template for summarizing notes created in a date range
    (format: YYYY-MM-DD). Returns the prompt text only — pair it with the
    notes://all Resource so the client has the notes to summarize."""
    return (
        f"Using the attached notes (see notes://all), summarize the ones "
        f"created between {start_date} and {end_date} in 2-3 sentences. "
        "Group related notes by theme if there is more than one."
    )
```

Run it: in the Inspector, read `notes://all` and confirm it lists every note you've added (including ones from Step 2's restart test); on a fresh database it reads `"No notes yet."`. Call `summarize_notes` with two dates and confirm you get back a filled-in prompt *string*, not an actual summary.

The Step 3 tests

Changed from Step 2: `test_db.py` adds a `list_all_notes` case; `test_server.py` adds a Resource case and a Prompt case.

```
# tests/unit/test_db.py (new case — add alongside Step 2's cases)
def test_list_all_notes():
    fresh_db()
    insert_note(DB, "one")
    insert_note(DB, "two")
    rows = list_all_notes(DB)
    result = "OK"
    if len(rows) != 2 or rows[0][1] != "one":
        result = "WRONG"
    print(f"case 4 (list all notes)          -> {result}")
    if os.path.exists(DB):
        os.remove(DB)
```

Add `list_all_notes` to the import at the top (`from src.db import find_notes, init_db, insert_note, list_all_notes`), and call `test_list_all_notes()` at the bottom.

```
# tests/integration/test_server.py (new cases — add before the end of main())
            # new in Step 3
            res = await session.read_resource("notes://all")
            show("case 5 (resource lists notes)",
                 "Buy milk" in res.contents[0].text)

            # new in Step 3
            pr = await session.get_prompt(
                "summarize_notes",
                {"start_date": "2026-09-01", "end_date": "2026-09-30"},
            )
            show("case 6 (prompt is a template)",
                 "2026-09-01" in pr.messages[0].content.text)
```

Run it (from the repo root):

```
python -m tests.unit.test_db
python -m tests.integration.test_server
```

Expected integration output (all six cases now):

```
case 1 (tools listed) -> OK
case 2 (add_note works) -> OK
case 3 (search, case-insensitive) -> OK
case 4 (empty note rejected) -> OK
```

```
case 5 (resource lists notes) -> OK
case 6 (prompt is a template) -> OK
```